

Prise de Décision Pacman

Rapport final sur les algorithmes de recherche et
stratégies réactives

Master 1 Informatique

Année Universitaire 2025–2026

Étudiant : Belhout Oussama

Table des matières

Résumé	1
1 Introduction	1
2 Partie I : Stratégies Réactives	1
2.1 Algorithme Glouton (Greedy)	1
3 Partie II : Recherche Arborescente	1
3.1 Minimax et Élagage Alpha-Bêta	2
4 Partie III : Architecture Technique	2
5 Conclusion	2

Résumé

Ce projet présente une étude de différents algorithmes de prise de décision pour un agent Pacman autonome. Nous avons implémenté et comparé des approches allant de simples réflexes gloutons à des recherches avancées dans l'arbre de jeu avec l'algorithme Minimax et l'élagage Alpha-Bêta.

1 Introduction

L'objectif principal est de permettre à l'agent Pacman de naviguer dans un labyrinthe, de collecter des pastilles et d'éviter les fantômes de manière autonome. Le problème est modélisé comme un processus de décision où chaque action doit maximiser le score tout en garantissant la survie.

2 Partie I : Stratégies Réactives

Les stratégies réactives choisissent une action basée uniquement sur l'état actuel de l'environnement, sans anticiper les coups futurs.

2.1 Algorithme Glouton (Greedy)

L'agent évalue chaque direction possible et choisit celle qui réduit le plus la distance vers la pastille la plus proche, tout en s'éloignant des fantômes immédiats.

```
1 def get_greedy_move(pacman_pos, pellets, ghosts):
2     best_move = None
3     max_score = -infinity
4
5     for move in valid_moves(pacman_pos):
6         # On calcule le score bas sur la proximit  de la
nourriture
7         score = -dist(move, nearest_pellet(pellets))
8
9         # Penalit  si un fant me est trop proche
10        for g in ghosts:
11            if dist(move, g) < 3:
12                score -= 500
13
14        if score > max_score:
15            max_score, best_move = score, move
16    return best_move
```

Listing 1 – Pseudo-code de la strat gie Gloutonne

3 Partie II : Recherche Arborescente

Pour am liorer les performances, nous utilisons des algorithmes de recherche qui simulent les futurs  tats possibles du jeu.

3.1 Minimax et Élagage Alpha-Bêta

L'algorithme Minimax modélise le jeu comme une compétition : Pacman cherche à maximiser son score, tandis que les fantômes cherchent à le minimiser. L'élagage Alpha-Bêta permet d'ignorer les branches inutiles de l'arbre pour accélérer le calcul.

```
1 def minimax(state, depth, alpha, beta, is_max):
2     if depth == 0 or game_over(state):
3         return evaluate(state)
4
5     if is_max: # Tour de Pacman
6         v = -infinity
7         for action in get_actions(state):
8             v = max(v, minimax(result(state, action), depth-1,
9 alpha, beta, False))
10            alpha = max(alpha, v)
11            if beta <= alpha: break # Coupure Alpha
12        return v
13    else: # Tour des Fantomes
14        v = +infinity
15        for action in get_ghost_actions(state):
16            v = min(v, minimax(result(state, action), depth-1,
17 alpha, beta, True))
18            beta = min(beta, v)
19            if beta <= alpha: break # Coupure Beta
20        return v
```

Listing 2 – Structure de l'algorithme Minimax

4 Partie III : Architecture Technique

Le système repose sur une séparation claire entre la logique de jeu et le moteur de décision :

- **Perception** : Récupération des positions (x, y) de tous les agents.
- **Évaluation** : Fonction de score pondérant la distance aux pastilles, le nombre de pastilles restantes et la proximité des fantômes.
- **Action** : Exécution du mouvement optimal calculé par l'IA.

5 Conclusion

Les tests montrent que l'algorithme Minimax avec élagage Alpha-Bêta offre une survie nettement supérieure aux stratégies réactives. Bien que plus gourmand en ressources, il permet d'anticiper les pièges tendus par les fantômes, rendant l'agent Pacman véritablement "intelligent".